amazon | SHOP • DELIVER • CREATE OPPORTUNITY

AMAZON-INSPIRED

E-COMMERCE SQL ANALYSIS

From Relational Data to Business Insights

PostgreSQL | SQL | Business Analysis

DIVYA BANSAL | ASPIRING DATA ANALYST

DATASET SOURCE

Brazilian E-Commerce Public Dataset by Olist

Source: Kaggle — olistbr/brazilian-ecommerce

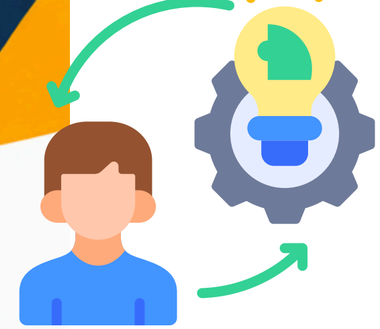
Portfolio analysis using public Olist data; not Amazon proprietary data.

DATA EXPLORATION
Understand the dataset and structure

14 BUSINESS QUESTIONS
Answer key questions with SQL

ADVANCED SQL ANALYSIS
Joins • CTEs • Window Functions • More

Turning data into better business decisions



02

PROJECT OVERVIEW

This project uses SQL to analyse the Brazilian Olist e-commerce dataset and turn relational data into insights on customer behaviour, sales performance, sellers and delivery.

PROJECT OBJECTIVE

- Explore e-commerce data with SQL.
- Answer 14 business questions across orders, customers, products, payments, reviews and sellers.

14 questions in the ER diagram

DATA OVERVIEW

- Multi-table relational dataset.
- Customers, orders, items, payments, reviews, products and sellers.
- Public Olist data, used as an Amazon Brazil-style case study.

5 tables (e.g. orders, customers, products)

KEY ANALYSIS AREAS

- Fulfilment and seller performance
- Payment methods and order value
- Customer spending and behaviour
- Category revenue and review scores
- State-level insights

5 core analysis areas

TOOLS & TECHNOLOGY

- PostgreSQL: storage and querying
- SQL: joins, CTEs, window functions
- VS Code: writing and testing queries

3 tools used

EXPECTED OUTCOMES

- Better understanding of customer and order behaviour
- Identify top categories, sellers and payment methods
- Support data-driven decisions

Actionable insights for business growth

KEY INSIGHT

A clear project scope turns a large dataset into **focused, answerable business questions.**

03 BUSINESS PROBLEM STATEMENTS

Turning data into smarter decisions for a better customer experience.

1. **Fulfilled orders by seller state**
   Where to direct regional fulfillment investment.
2. **Cumulative revenue by category**
   How revenue builds for flash-sale planning
3. **Payment methods and order value**
   Input for payment-provider talks.
4. **Highest-spending customer**
   Starting point for a premium loyalty tier.
5. **Review score by category**
   Which categories need attention first.
6. **Orders per customer and state**
   State-level campaign planning
7. **Sellers with no delivered orders**
   Re-engagement list of dormant sellers.
8. **Top 5 categories by revenue**
   Homepage and promotion priorities.
9. **Median delivery time**
   Typical customer wait, robust to outliers.
10. **Products never ordered**
    Quarterly catalogue clean-up
11. **Sellers above average**
    Invite list for a top-seller badge
12. **State-level review scores**
    Last-mile experience by state
13. **Customers who never reviewed**
    Targets for review reminders.
14. **Month with the most orders**
    Peak-demand planning

KEY INSIGHT
Each question is tied to a team decision, so every query has a **clear business purpose.**

04

MAIN TABLES AND PROFILING COUNTS

orders

99,441

DATASET PROFILING RESULT

One row per order

order_items

112,650

DATASET PROFILING RESULT

One row per item in an order

order_payments

103,886

DATASET PROFILING RESULT

One row per payment record

order_reviews

99,224

DATASET PROFILING RESULT

One row per review

products

32,951

DATASET PROFILING RESULT

One row per product

customers

99K+

One row per customer record

sellers

3K+

One row per registered seller

product_category

70+

Category name lookup

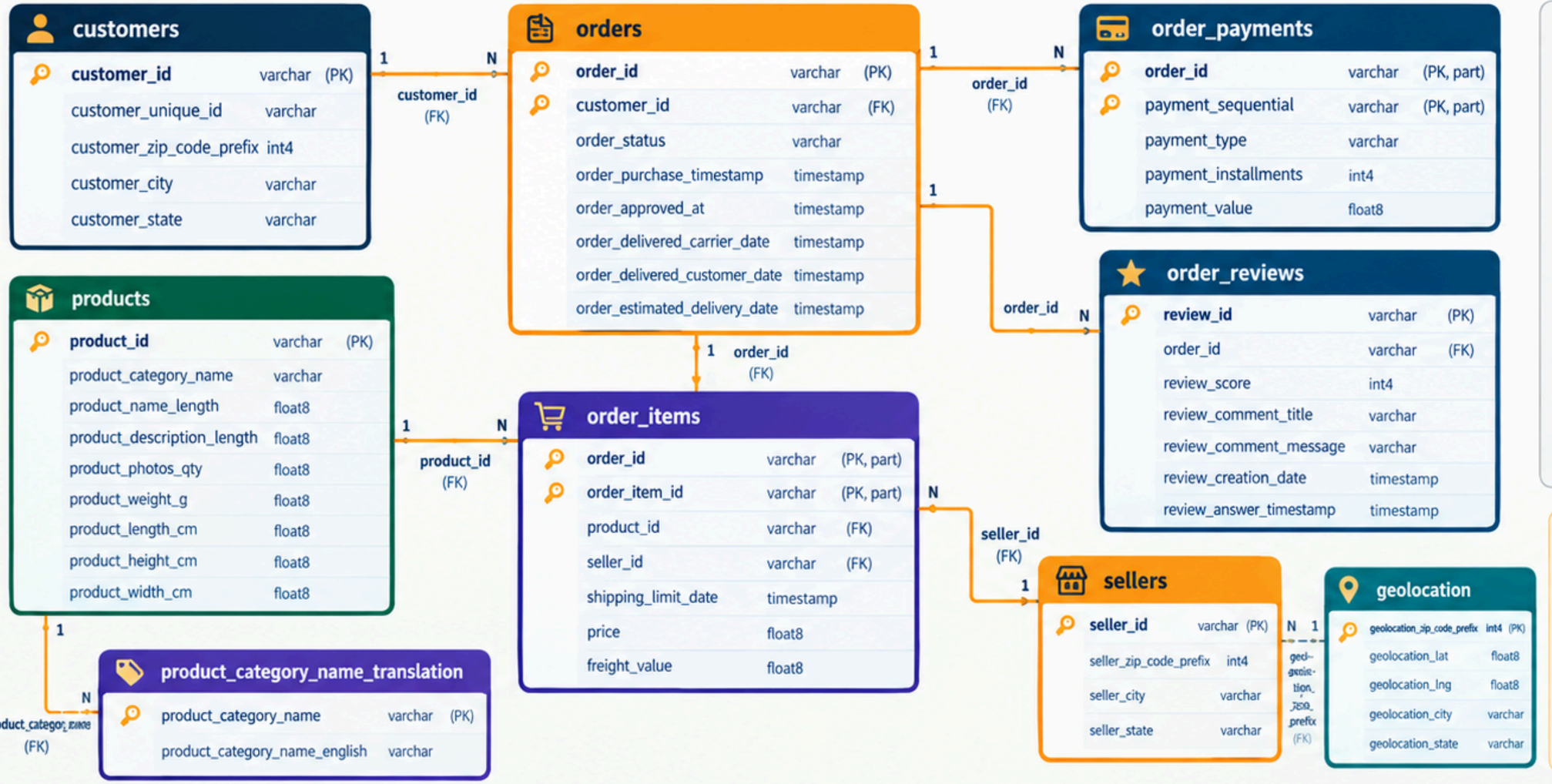
KEY INSIGHT

Table size is only the starting point; understanding what each row represents is essential.

Amazon E-Commerce | Entity Relationship Diagram

PostgreSQL • Database Schema • Key Relationships

Better Data
Better Decisions
Stronger E-Commerce



Key Relationships

1. customers → orders (customer_id)
2. orders → order_items (order_id)
3. orders → order_payments (order_id)
4. orders → order_reviews (order_id)
5. order_items → products (product_id)
6. order_items → sellers (seller_id)
7. products → product_category_name_translation (product_category_name)
8. sellers → geolocation (seller_zip_code_prefix)

Important: Data Grain

- **Orders**, items, payments and reviews have different grains.
- An **order** can contain multiple items, sellers, payments and potentially reviews.
- Use the correct **joins** and aggregation **level** to avoid duplicate rows and incorrect totals.

Legend: = Primary Key (PK) | FK = Foreign Key (FK) | = One to Many | = Lookup Relationship (not enforced FK)

KEY INSIGHT

Correct joins depend on knowing the relationship and grain of each table.

06

DATA PROFILING AND QUALITY CHECKS

- ✓ Row counts and uniqueness
- ✓ Null values in key columns
- ✓ Foreign-key matching
- ✓ Valid order statuses
- ✓ Purchase and delivery timestamp checks
- ✓ Missing product-category values
- ✓ Possible chronological inconsistencies

VERIFIED FINDING

610

products had missing category names

Cause not inferred without evidence.

KEY INSIGHT

Data-quality checks help prevent misleading conclusions later in the analysis.

07 DEFINITIONS APPLIED IN THE AUDIT

Fulfilled order

```
order_status = 'delivered'
```

Product-price revenue

```
SUM(order_items.price), shipping excluded
```

Real customer

```
customer_unique_id
```

Distinct order counts

```
COUNT(DISTINCT order_id) when joins can repeat rows
```

Median delivery duration

```
PERCENTILE_CONT(0.50)
```

Seller benchmark

Average delivered orders across all registered sellers, including zero-order sellers

Adjust these definitions if a business stakeholder defines a metric differently.

METRIC DEFINITIONS AND ASSUMPTIONS

KEY INSIGHT

Consistent metric definitions make comparisons meaningful.

Q1 ORDERS BY SELLER STATE

Which seller states have the most fulfilled orders?

SQL QUERY

```
SELECT sellers.seller_state,
    COUNT(DISTINCT orders.order_id) as
delivered_orders
FROM sellers
INNER JOIN order_items
  ON order_items.seller_id = sellers.seller_id
INNER JOIN orders
  ON orders.order_id = order_items.order_id
WHERE orders.order_status = 'delivered'
GROUP BY sellers.seller_state,
     orders.order_status
ORDER BY delivered_orders DESC;
```

RESULT PREVIEW

	seller_state	delivered_orders
1	SP	68641
2	MG	7735
3	PR	7512
4	RJ	4227
5	SC	3603
6	RS	1962
7	DF	808
8	BA	550
9	GO	451
10	PE	403
11	MA	389
12	ES	310
13	MT	136

ANALYSIS NOTES

1. COUNT(DISTINCT order_id) counts each order once per state, even with several items.
2. Delivered status is the fulfilment definition used throughout.
3. An order with sellers in two states counts once in each state.

KEY INSIGHT

Seller-state comparisons should count delivered orders once per seller and apply the same fulfillment definition consistently.

Q2 CUMULATIVE REVENUE BY CATEGORY

For each product category, calculate the cumulative revenue generated as orders come in over time.

SQL QUERY

```
WITH revenue AS
  (SELECT pc.product_category_name_english AS
product_category,
      o.order_purchase_timestamp,
      sum(oi.price) as rev
   FROM products p
   INNER JOIN product_category_name_translation pc ON
pc.product_category_name = p.product_category_name
   INNER JOIN order_items oi ON oi.product_id = p.product_id
   INNER JOIN orders o ON o.order_id = oi.order_id
   WHERE o.order_status = 'delivered'
   GROUP BY pc.product_category_name_english,
       o.order_purchase_timestamp)
SELECT product_category,
    order_purchase_timestamp,
    SUM(rev) OVER(PARTITION BY product_category
           ORDER BY order_purchase_timestamp ROWS BETWEEN
UNBOUNDED PRECEDING AND CURRENT ROW) cumulative_revenue
FROM revenue
ORDER BY product_category,
     order_purchase_timestamp;
```

RESULT PREVIEW

	product_category	order_purchase_timestamp	cumulative_revenue
1	agro_industry_and_commerce	2017-01-23 07:03:00	43.98
2	agro_industry_and_commerce	2017-01-31 17:33:00	65.97
3	agro_industry_and_commerce	2017-02-05 19:35:00	87.96
4	agro_industry_and_commerce	2017-02-08 02:32:00	109.94999999999999
5	agro_industry_and_commerce	2017-02-12 20:42:00	202.85
6	agro_industry_and_commerce	2017-02-13 13:00:00	224.84
7	agro_industry_and_commerce	2017-02-16 10:24:00	246.83
8	agro_industry_and_commerce	2017-02-18 16:54:00	268.82
9	agro_industry_and_commerce	2017-02-21 14:25:00	290.81
10	agro_industry_and_commerce	2017-03-17 15:57:00	350.8
11	agro_industry_and_commerce	2017-03-20 18:22:00	372.8

KEY INSIGHT

Cumulative revenue shows how each category's sales value builds over time.

Q3

PAYMENT METHOD AND AVERAGE ORDER VALUE

Which payment method do customers use the most, and what is the average order value for each payment type?

SQL QUERY

```
WITH order_totals AS
  (SELECT order_id,
      SUM(payment_value) order_value
   FROM order_payments
   GROUP BY order_id),

order_payment_types AS
  (SELECT DISTINCT order_id,
          payment_type
   FROM order_payments)

SELECT opt.payment_type,
    COUNT(*) AS orders_count,
    ROUND(AVG(ot.order_value)::NUMERIC, 2) AS
avg_order_value
FROM order_payment_types opt
INNER JOIN order_totals ot ON opt.order_id = ot.order_id
GROUP BY opt.payment_type
ORDER BY orders_count DESC;
```

RESULT PREVIEW

payment_type	orders_count	avg_order_value
credit_card	76505	166.47
boleto	19784	145.03
voucher	3866	135.58
debit_card	1528	142.73
not_defined	3	0.00

ANALYSIS NOTES

1. Average paid by one type is not the average full order value.
2. Sum payments per order first, then one row per order and type.
3. Split-payment orders appear under each type used.

KEY INSIGHT

Payment metrics must distinguish partial payment amounts from full order values.

Q4 HIGHEST-SPENDING CUSTOMER

Find the customer who has spent the most money across all their orders.

SQL QUERY

```
SELECT
  c.customer_unique_id,
  SUM(op.payment_value) total_amount
FROM customers c
INNER JOIN
orders o
ON c.customer_id = o.customer_id
INNER JOIN
order_payments op
ON op.order_id = o.order_id
GROUP BY
c.customer_unique_id
ORDER BY total_amount DESC LIMIT 1 ;
```

RESULT PREVIEW

customer_unique_id	total_amount
0a0a92112bd4c708ca5fde585afaa872	13664.08

ANALYSIS NOTES

1. Measures recorded payment value, not net revenue after refunds.
2. customer_unique_id merges multiple customer_id records of one person.
3. Check for ties at first place before naming one customer.

KEY INSIGHT

Customer identity must be consistent when combining spending across multiple customer records.

Q5 REVIEW SCORE BY CATEGORY

Find the average review score for each product category.

SQL QUERY

```
With DISTINCT_order AS
  (SELECT DISTINCT o.order_id,
          pc.product_category_name_english AS
product_category,
          ore.review_score
   FROM products p
   INNER JOIN product_category_name_translation pc ON
pc.product_category_name = p.product_category_name
   INNER JOIN order_items oi ON oi.product_id = p.product_id
   INNER JOIN orders o ON o.order_id = oi.order_id
   INNER JOIN order_reviews ore ON ore.order_id = o.order_id)
SELECT product_category,
    ROUND(AVG(review_score), 2) as avg_review_score
FROM DISTINCT_order
GROUP BY product_category
ORDER BY avg_review_score DESC;
```

RESULT PREVIEW

product_category	avg_review_score
cds_dvds_musicals	4.67
fashion_childrens_clothes	4.50
books_general_interest	4.46
costruction_tools_tools	4.43
books_technical	4.40
flowers	4.39
food_drink	4.38
luggage_accessories	4.33

KEY INSIGHT

A review score associated with an order is a proxy for category experience, not necessarily a direct product rating.

Q6 ORDERS PER CUSTOMER AND STATE

Find the total number of orders placed by each customer, broken down by the state they live in.

SQL QUERY

```
SELECT c.customer_unique_id,
    c.customer_state,
    COUNT(DISTINCT o.order_id) as
total_no_of_orders
FROM customers c
INNER JOIN orders o ON c.customer_id =
o.customer_id
GROUP BY c.customer_unique_id,
     c.customer_state
ORDER BY total_no_of_orders DESC,
     c.customer_state;
```

RESULT PREVIEW

customer_unique_id	customer_state	total_no_of_orders
8d50f5eadf50201ccdcedfb9e2ac8455	SP	17
3e43e6105506432c953e165fb2acf44c	SP	9
1b6c7548a2a1f9037c1fd3ddfed95f33	MG	7
ca77025e7201e3b30c44b472ff346268	PE	7
6469f99c1f9dfae7733b25662e7f1782	SP	7
f0e310a6839dce9de1638e0fe5ab282a	ES	6
dc813062e0fc23409cd255f7f53c7074	PE	6
12f5d6e1cbf93dafd9dcc19095df0b3d	PR	6

KEY INSIGHT

Choosing the correct customer identifier changes whether repeat purchases are combined across records.

Q7 SELLERS WITH NO DELIVERED ORDERS

Identify sellers who registered on the platform but have never fulfilled a single order.

SQL QUERY

```
SELECT s.seller_id,
    s.seller_state
FROM sellers s
WHERE NOT EXISTS
    (SELECT 1
     FROM order_items oi
     INNER JOIN orders o USING(order_id)
     WHERE oi.seller_id = s.seller_id
       AND o.order_status = 'delivered')
ORDER BY s.seller_state;
```

RESULT PREVIEW

seller_id	seller_state
4be2e7f96b4fd749d52dff41f80e39dd	AC
f3055483058c84d75ca2a3314fb4deef	BA
cf281cf8b7affbdfe751b29cc85580e1	CE
61f159ef6da2d441951d2c0efa719362	ES
578460d8319c51dab351cd9db95eadfe	GO
9b1050e85becf3ae99c1cf4934c9bc5d	MG
ad8ada0ffa993cb1197d102d9ab21a3e	MG
fa18c782a4866d313cadab567097f8f1	MG

ANALYSIS NOTES

1. NOT EXISTS keeps sellers with no delivered-order item at all.
2. Includes sellers whose orders were never delivered and sellers with no items.
3. Split the list by reason before any re-engagement campaign.

KEY INSIGHT

A seller with no delivered orders is not necessarily a seller with no order history.

Q8

TOP FIVE CATEGORIES BY REVENUE

Find the top 5 product categories by total revenue.

SQL QUERY

```
SELECT
    pc.product_category_name_english
product_category,
    ROUND(SUM(oi.price)::NUMERIC,2) total_revenue
FROM product_category_name_translation pc
INNER JOIN products
    USING(product_category_name)
INNER JOIN order_items oi
    USING(product_id)
INNER JOIN orders o
  ON oi.order_id = o.order_id
WHERE o.order_status = 'delivered'
GROUP BY pc.product_category_name_english
ORDER BY total_revenue DESC, product_category
LIMIT 5;
```

RESULT PREVIEW

product_category	total_revenue
health_beauty	1233131.72
watches_gifts	1166176.98
bed_bath_table	1023434.76
sports_leisure	954852.55
computers_accessories	888724.61

ANALYSIS NOTES

1. Revenue = SUM(price) from delivered orders; freight excluded.
2. Used Inner Join as product with revnenue is only needed
3. Review a NULL category rather than hiding it.

KEY INSIGHT

The top-category ranking depends on the revenue definition and which orders are included.

Q9

MEDIAN DELIVERY TIME

Find the median delivery time (in days) between order placement and actual delivery.

SQL QUERY

```
With delivery_time AS
  (SELECT EXTRACT(EPOCH
          FROM (order_delivered_customer_date -
order_purchase_timestamp)) / 86400.0 AS
delivery_days
   FROM orders
   WHERE order_status = 'delivered'
     AND order_delivered_customer_date IS NOT NULL
     AND order_purchase_timestamp IS NOT NULL)

SELECT ROUND(PERCENTILE_CONT(0.50) WITHIN GROUP (
                          ORDER BY
delivery_days)::NUMERIC, 2) median_delivery_days
FROM delivery_time;
```

RESULT PREVIEW

median_delivery_days
10.22

ANALYSIS NOTES

1. The median is less affected by extreme delays than the mean.
2. EXTRACT(EPOCH ...) / 86400 converts the interval to numeric days.
3. Investigate negative durations before excluding them.

KEY INSIGHT

The median describes the midpoint of delivery durations and is less influenced by extreme values than the mean.

Q10

PRODUCTS NEVER ORDERED

Find all products that have never been ordered.

SQL QUERY

```
SELECT product_id,
    product_category_name
FROM products
WHERE NOT EXISTS
    (SELECT 1
     FROM order_items
     WHERE products.product_id =
order_items.product_id)
ORDER BY products.product_id;
```

RESULT PREVIEW

product_id	product_c…

There is no such product that have never been ordered.

ANALYSIS NOTES

1. A product counts as ordered if it is in any order item, whatever the status.
2. NOT EXISTS stops at the first match for each product.
3. Output is product_id and category for catalogue review.

KEY INSIGHT

Catalog existence does not guarantee that a product has appeared in an order.

Q11

SELLERS ABOVE AVERAGE

DIVYA BANSAL | POSTGRESQL ANALYSIS

Find sellers who have fulfilled more orders than the average seller on the platform.

SQL QUERY

```
WITH sellers_orders AS
  (SELECT s.seller_id,
      COUNT(DISTINCT o.order_id) AS delivered_orders
   FROM sellers s
   LEFT JOIN order_items oi ON s.seller_id =
oi.seller_id
   LEFT JOIN orders o ON oi.order_id = o.order_id
   AND o.order_status = 'delivered'
   GROUP BY s.seller_id),
   avg_orders AS
  (SELECT AVG(delivered_orders) AS avg_delivered_orders
   FROM sellers_orders)
SELECT seller_id,
    delivered_orders,
    ROUND(avg_delivered_orders, 2) avg_delivered_orders
FROM sellers_orders
CROSS JOIN avg_orders ao
WHERE delivered_orders > avg_delivered_orders
ORDER BY delivered_orders DESC;
```

KEY INSIGHT

A fair performance benchmark must define the seller population and include zero-order sellers when intended.

RESULT PREVIEW

seller_id	delivered_orders	avg_delivered_orders
6560211a19b47992c3666cc44a7e94c0	1819	31.61
4a3ca9315b744ce9f8e9374361493884	1772	31.61
cc419e0650a3c5ba77189a1882b7556a	1651	31.61
1f50f920176fa81dab994f9023523100	1399	31.61
da8622b14eb17ae2831f4ac5b9dab84a	1311	31.61
955fee9216a65b617aa5c0531780ce60	1261	31.61
7a67c85e85bb2ce8582c35f2203ad736	1145	31.61

ANALYSIS NOTES

1. LEFT JOIN with the delivered condition inside ON keeps zero-order sellers.
2. COUNT(DISTINCT ...) stops multi-item orders inflating counts.
3. CROSS JOIN attaches the single average row to every seller.

Q12

STATE-LEVEL REVIEW SCORES

Find which Brazilian states have the highest average customer review score for orders delivered there.

SQL QUERY

```
SELECT c.customer_state,
    ROUND(AVG(ore.review_score), 2) as
avg_customer_review
FROM customers c
INNER JOIN orders o USING(customer_id)
INNER JOIN order_reviews ore USING(order_id)
WHERE ore.review_score IS NOT NULL
AND o.order_status = 'delivered'
GROUP BY c.customer_state
ORDER BY avg_customer_review DESC;
```

RESULT PREVIEW

customer_state	avg_customer_review
AP	4.19
PR	4.18
AM	4.18
SP	4.17
MG	4.14
RS	4.13
MS	4.12
...	...

ANALYSIS NOTES

1. One score per order first, so multiple reviews do not overweight an order.
2. Delivered orders only, as we are focusing where review score is not null
3. State is the customer state, not necessarily seller or route.

KEY INSIGHT

State-level comparisons are only meaningful when the geographic field and review weighting are clearly defined.

Q13

CUSTOMERS WHO NEVER REVIEWED

Identify customers who have placed orders but never left a review.

SQL QUERY

```
SELECT DISTINCT c.customer_unique_id
FROM customers c
INNER JOIN orders o USING(customer_id)
WHERE NOT EXISTS
        (SELECT 1
         FROM customers c1
         INNER JOIN orders o2 USING
(customer_id)
         INNER JOIN order_reviews ore USING
(order_id)
         WHERE c1.customer_unique_id =
c.customer_unique_id) ;
```

RESULT PREVIEW

customer_unique_id
528b011eb7fab3d59c336cc7248eed3a
ea870f4fdfd85ac98ab775b76efe3143
367f4686d7112d69feed92b02a1775ed
c56d066e503008b8d0bf4204857c588d
f329e0e1d8897ffa466a836ed216a23b
ec85abbbac44c1bb668a04903d033b6d
dfe0f45b0b8cf0ce9c4cfb41c0808240

ANALYSIS NOTES

1. Correlated NOT EXISTS checks every order of the same real customer.
2. Checking orders one by one flags people who reviewed another order.
3. customer_unique_id spans multiple customer_id records.

KEY INSIGHT

An unreviewed order is not the same as a customer who has never reviewed any order.

Q14

MONTH WITH THE MOST ORDERS

Find the month with the highest number of orders placed across the entire platform.

SQL QUERY

```
WITH monthly_orders AS
  (SELECT DATE_TRUNC('month',
order_purchase_timestamp) AS month_start,
      COUNT(*) AS total_orders
   FROM orders
   WHERE order_purchase_timestamp IS NOT NULL
   GROUP BY DATE_TRUNC('month',
order_purchase_timestamp))
SELECT TO_CHAR(month_start, 'Month YYYY') AS
month_name,
    total_orders
FROM monthly_orders
ORDER BY total_orders DESC
LIMIT 1;
```

RESULT PREVIEW

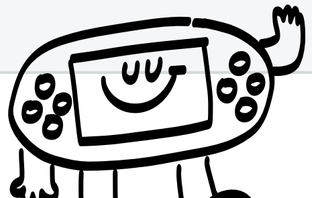
month_name	total_orders
November 2017	7544

ANALYSIS NOTES

1. DATE_TRUNC('month', ...) keeps year and month together.
2. COUNT(*) counts order rows; orders has one row per order.
3. Check for ties at the top before naming one month.

KEY INSIGHT

Grouping by both month and year prevents different years' January orders from being combined incorrectly.



25 CHALLENGE → DECISION → WHY IT MATTERS

Challenge	Decision	Why it matters
Join fan-out: items × payments	Aggregate payments per order first	Prevents inflated totals
Counting item rows	COUNT(DISTINCT order_id)	One order is counted once
Zero-order sellers dropped from average	LEFT JOIN from all sellers	Benchmark covers the full population
customer_id vs customer_unique_id	Use customer_unique_id for people	Repeat buyers are combined
Median confused with mean	PERCENTILE_CONT for delivery time	Outliers do not skew the midpoint
Unreviewed order ≠ non-reviewing customer	Check all orders of the customer	Avoids false targeting
Missing categories / translations	LEFT JOIN and report NULLs	Revenue stays visible, not lost

KEY INSIGHT

Small SQL design decisions can materially change a business conclusion.

amazon | Business Insights & Recommendations

Turning SQL Analysis into Business Decisions

Better Data
Better Decisions
Stronger E-Commerce

1. Sales & Revenue

Analysis

- Monthly order volume
- Cumulative category revenue
- Top five categories by product-price revenue

Business Use

- Identify high-contributing categories and periods that deserve further investigation.

Result (from SQL)

[Insert verified category and month results]

2. Customer Behaviour

Analysis

- Spending by real customer identifier
- Order frequency by state
- Customers who have never submitted a review

Business Use

- Explore repeat-purchase patterns and possible opportunities to improve customer engagement.

Result (from SQL)

[Insert verified customer findings]

3. Seller Performance

Analysis

- Delivered orders by seller state
- Sellers with no delivered orders
- Sellers above the average delivered-order benchmark

Business Use

- Identify performance differences and sellers who may need operational follow-up.

Result (from SQL)

[Insert verified seller findings]

4. Customer Experience

Analysis

- Median delivery duration
- Average review scores by category and customer state

Business Use

- Investigate delivery delays and areas where customer experience may need attention.

Result (from SQL)

[Insert verified delivery and review findings]

5. Payments & Product Coverage

Analysis

- Payment-method usage
- Average order value by payment type
- Products absent from order items

Business Use

- Understand payment patterns and review products that have never appeared in an order.

Result (from SQL)

[Insert verified payment and product findings]

KEY INSIGHT

SQL becomes valuable when reliable metrics are translated into specific business questions and follow-up actions.

KEY LEARNINGS AND PORTFOLIO CLOSE

28

WHAT THIS PROJECT TAUGHT ME

- Understanding table grain and relationships
- Profiling data before analysis
- Writing SQL for practical business questions
- Preventing duplicate counting and join fan-out
- Using CTEs, window functions, subqueries, percentiles
- Validating assumptions before communicating findings

PostgreSQL | SQL
CTEs | Window Functions
Data Profiling | Business Analysis

GitHub: URL]https://github.com/divyabansal5509

“A query can execute successfully and still answer the wrong business question. Defining the metric is part of the analysis.”

KEY INSIGHT

Writing a query is only one part of analysis. Understanding what the result means is what makes it valuable.

THANK YOU!

Thank you for your time. I hope this analysis shows how SQL can turn relational data into business insight.

Presented by Divya Bansal | Aspiring Data Analyst

KEY INSIGHT

Good analysis goes beyond writing queries.
It starts with asking the right business questions, validating the data, and turning SQL results into meaningful insights.